

---

# Hyperparameter Analysis of Linear SVM on Synthetic Data

---

## Group Members:

Shresthraj(240996)

Sana(240924)

Praveen(240790)

Virendra(241172)

Yash(241195)

Deepak(240327)

## Abstract

In this report we study the effect of hyperparameters on a Linear Support Vector Machine (SVM) using synthetic datasets generated by the provided code. Experiments are performed across different challenge levels and dataset sizes to analyze the influence of parameters such as  $C$ , tolerance, and maximum iterations on training time and classification accuracy.

## 1 Introduction

Support Vector Machines (SVMs) are widely used linear classifiers that find a hyperplane separating two classes with maximum margin. The performance of SVM models depends strongly on hyperparameters such as the regularization parameter  $C$ , tolerance, and optimization iterations. In this assignment we analyze the behavior of a LinearSVC classifier on synthetic 2D datasets generated using the provided data generation code.

## 2 Problem 1: Linear Separability

**Answer: Yes.** The dataset generated by the function `genChallengeData(level, n)` is guaranteed to be linearly separable for all valid values of the parameters `level` and `n`.

In this section, we examine the geometric bounds of the data generation process, provide a rigorous mathematical proof of its linear separability, and verify our reasoning experimentally using visualizations of a linear SVM classifier.

### 2.1 Structure of the Generated Dataset

The function `genChallengeData` generates a two-dimensional binary classification dataset consisting of two positive clusters and one negative cluster. Let us define the constants from the code:

- Let the spatial constant  $v = \sqrt{2}$ .
- Let the level-dependent scaling factor be  $\alpha = 1 + \frac{1}{2 \cdot \text{level}}$ .

Because the problem dictates that the challenge `level` must be strictly greater than zero, it mathematically follows that the scaling factor is bounded:  $\alpha > 1$ .

The exact cluster centers are defined as:

$$\begin{aligned}\mu_{Pos1} &= (-v\alpha, v) \\ \mu_{Pos2} &= (v\alpha, -v) \\ \mu_{Neg} &= (-v\alpha, -v)\end{aligned}$$

Each cluster generates data points strictly within a circular region (sphere in 2D) of radius  $r = 1$  using the helper function `genSphericalData`.

## 2.2 Rigorous Mathematical Proof of Separability

Two classes are strictly linearly separable if there exists a hyperplane (a line in 2D) such that the perpendicular distance from the line to the center of every cluster is strictly greater than the radius of those clusters ( $r = 1$ ). If this condition is met, no data points from opposing classes can ever cross the decision boundary.

### Step 1: Propose a Decision Boundary

Consider the line that passes exactly halfway between the negative center and the two positive centers. The midpoint between  $\mu_{Neg}$  and  $\mu_{Pos1}$  is  $M_1 = (-v\alpha, 0)$ . The midpoint between  $\mu_{Neg}$  and  $\mu_{Pos2}$  is  $M_2 = (0, -v)$ .

Using the intercept form, the equation of the line passing through  $M_1$  and  $M_2$  is:

$$\begin{aligned}\frac{x}{-v\alpha} + \frac{y}{-v} &= 1 \\ x + \alpha y + v\alpha &= 0\end{aligned}$$

### Step 2: Evaluate Center Distances and Class Sides

We calculate the perpendicular distance  $d$  from each cluster center to this proposed decision boundary using the standard point-to-line distance formula  $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$ , where  $A = 1$ ,  $B = \alpha$ , and  $C = v\alpha$ .

Distance to the first positive center  $\mu_{Pos1}(-v\alpha, v)$ :

$$d_{P1} = \frac{|-v\alpha + \alpha(v) + v\alpha|}{\sqrt{1^2 + \alpha^2}} = \frac{|v\alpha|}{\sqrt{1 + \alpha^2}} = \frac{v\alpha}{\sqrt{1 + \alpha^2}}$$

Evaluating the line equation without the absolute value yields  $+v\alpha$ , placing it strictly on the positive side of the boundary.

Distance to the second positive center  $\mu_{Pos2}(v\alpha, -v)$ :

$$d_{P2} = \frac{|v\alpha + \alpha(-v) + v\alpha|}{\sqrt{1^2 + \alpha^2}} = \frac{|v\alpha|}{\sqrt{1 + \alpha^2}} = \frac{v\alpha}{\sqrt{1 + \alpha^2}}$$

Evaluating the line equation yields  $+v\alpha$ , also placing it strictly on the positive side.

Distance to the negative center  $\mu_{Neg}(-v\alpha, -v)$ :

$$d_N = \frac{|-v\alpha + \alpha(-v) + v\alpha|}{\sqrt{1^2 + \alpha^2}} = \frac{|-v\alpha|}{\sqrt{1 + \alpha^2}} = \frac{v\alpha}{\sqrt{1 + \alpha^2}}$$

Evaluating the line equation yields  $-v\alpha$ , placing it strictly on the negative side.

The line successfully separates the positive centers from the negative center. The absolute distance from the decision boundary to the center of every cluster is identically  $d = \frac{v\alpha}{\sqrt{1 + \alpha^2}}$ .

### Step 3: Prove the Hard Margin Condition ( $d > r$ )

For strict separability, the distance  $d$  must be strictly greater than the cluster radius  $r = 1$ . Let us evaluate  $d^2$ , noting that  $v = \sqrt{2}$ , so  $v^2 = 2$ :

$$\begin{aligned}d^2 &= \frac{v^2\alpha^2}{1 + \alpha^2} \\ d^2 &= \frac{2\alpha^2}{1 + \alpha^2} \\ d^2 &= \frac{\alpha^2 + \alpha^2}{1 + \alpha^2}\end{aligned}$$

From our initial definition, we established that  $\alpha > 1$ , which means  $\alpha^2 > 1$ . Therefore, substituting 1 with  $\alpha^2$  in the denominator makes the fraction strictly smaller:

$$\frac{\alpha^2 + \alpha^2}{1 + \alpha^2} > \frac{\alpha^2 + 1}{1 + \alpha^2} = 1$$

Since  $d^2 > 1$ , it logically follows that  $d > 1$ .

Because the perpendicular distance from the optimal decision boundary to all cluster centers is strictly greater than the maximum extent (radius) of the data points, the clusters will never intersect the boundary line.

### 2.3 Effect of Parameters $n$ and $level$

The parameter  $n$  determines the number of points sampled. Because the data is generated strictly within the bounds of the  $r = 1$  radius, the support of the distributions for the positive and negative classes are fundamentally disjoint. Sampling any arbitrary number of points  $n$  will only change the density within these bounds, leaving the linear separability completely unaffected.

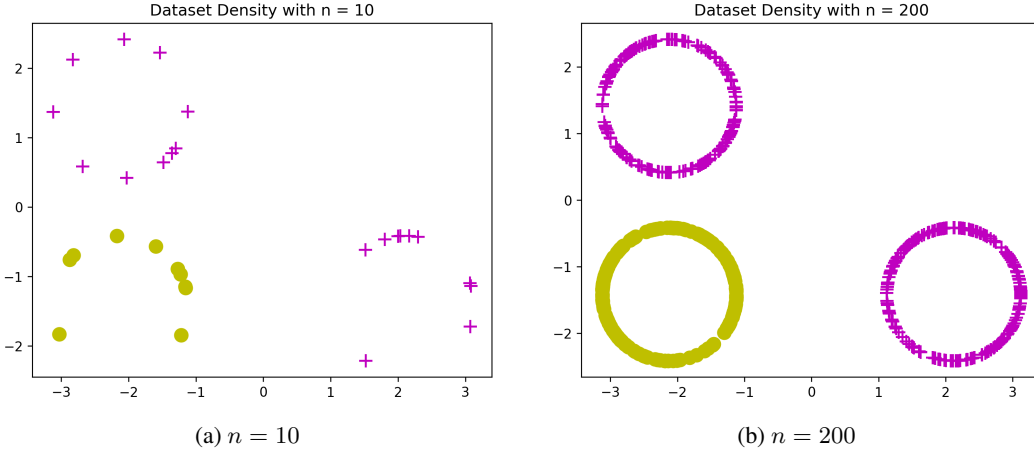


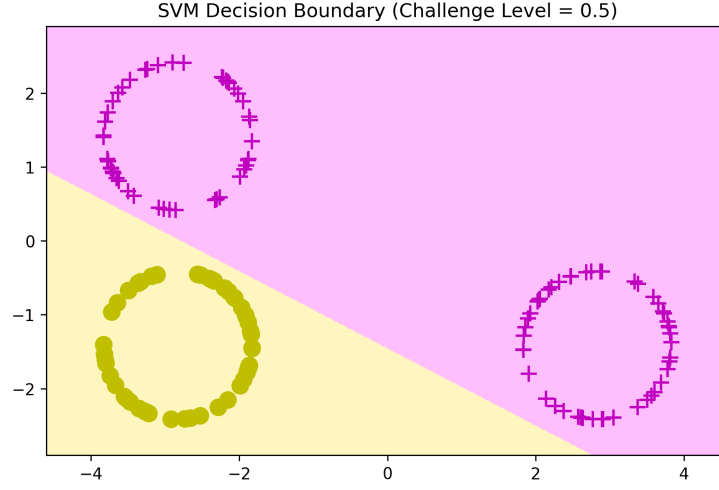
Figure 1: Effect of varying the number of samples  $n$  at a fixed challenge level. The density changes, but the margin remains intact.

The parameter  $level$  controls the horizontal spacing. As  $level \rightarrow \infty$ , the scaling factor  $\alpha \rightarrow 1$ . Even at an infinitely large challenge level, the closest bounds of the opposing clusters never touch.  $\mu_{Pos1}$  and  $\mu_{Neg}$  are vertically aligned at  $x = -\sqrt{2}$  with  $y$ -coordinates of  $\sqrt{2}$  and  $-\sqrt{2}$  respectively. The distance between these centers is  $2\sqrt{2} \approx 2.828$ . Since this center-to-center distance is strictly greater than the combined radii ( $2r = 2$ ), an overlap is mathematically impossible.

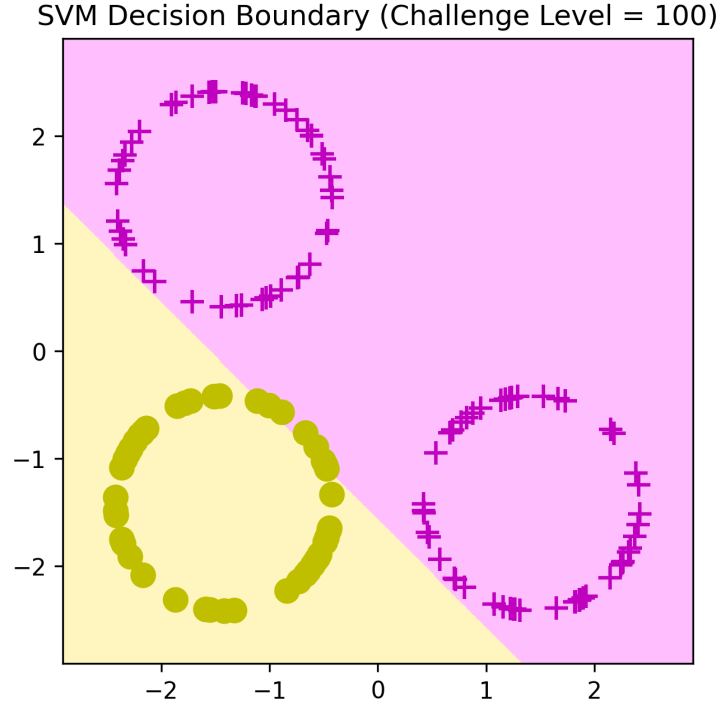
### 2.4 Experimental Visualization

To visually confirm our proof, we generated datasets for increasing values of  $level$  while keeping  $n = 50$  fixed, and trained a linear SVM classifier on each.

*Note: At extremely high challenge levels (e.g.,  $level = 100$ ), the mathematical margin between the classes becomes narrower than the physical rendering size of standard plot markers. While the data remains mathematically strictly separable (as proven in Section 2.2), the rendered scatter plot visually appears to overlap due to the physical thickness of the plotted ink.*



(a) Small Challenge Level (level = 0.5)



(b) Large Challenge Level (level = 100)

Figure 2: Datasets generated for different challenge levels. The linear SVM consistently finds a valid decision boundary, even at extremely high challenge levels like level = 100, confirming our mathematical proof that the classes never overlap.

## 2.5 Conclusion

Based on the rigorous mathematical proof and supported by our experimental visualizations, we conclude that the dataset generated by `genChallengeData` is **guaranteed to be linearly separable for all valid values of level and  $n$ .**

### 3 Problem 2: Small Challenge Level

#### 3.1 Experimental Procedure

To analyze the behavior of the LinearSVC classifier for small challenge levels, we conducted a series of experiments using datasets generated by the `genChallengeData` function with `level < 0.5`. At such low challenge levels, the generated data points from the two classes are relatively well separated, so the classification task is expected to be easier than in the large challenge setting.

For each experiment, a dataset with a fixed number of samples was generated while maintaining a small challenge level. The LinearSVC classifier from the scikit-learn library was then trained on this dataset.

We first used a binary-search-based procedure to determine the smallest value of the regularization parameter  $C$  that still achieved 100% training accuracy. After identifying this threshold value of  $C$ , we fixed it and then varied `max_iter` to determine the smallest number of iterations required for perfect classification. Finally, after fixing both  $C$  and `max_iter`, we varied the tolerance parameter `tol` to study the largest tolerance value that still preserved 100% training accuracy.

The experiments were implemented using Python loops that evaluated multiple combinations of these hyperparameters. For each configuration, the classifier was trained and the training accuracy was recorded. If the classifier failed to achieve perfect accuracy, the value of  $C$  was adjusted using binary search, or `max_iter` was increased, or the tolerance parameter was modified accordingly.

Using this experimental procedure, we identified the minimum hyperparameter values required for the model to correctly classify all training samples when the challenge level of the dataset is small.

#### 3.2 Objective

In this experiment, we work with the *easy* synthetic dataset (small challenge level). Specifically, we set the challenge level to a value  $< 0.5$  (e.g. `level = 0.3`) and generate  $n = 100$  points from each class using `genChallengeData`. The task is to find the **smallest** regularization constant  $C$  and **smallest** `max_iter`, along with the **largest** `tol`, for which the linear SVM still achieves 100% training accuracy. We systematically explore different values of  $C$ , `max_iter`, and `tol` to determine these minimal settings.

#### 3.3 Dataset Generation and Visualization

We use the provided data generator with `level = 0.3` and  $n = 100$ . The function returns positive and negative class samples, which we combine into a single dataset. Figure 1 shows a scatter plot of the generated data.

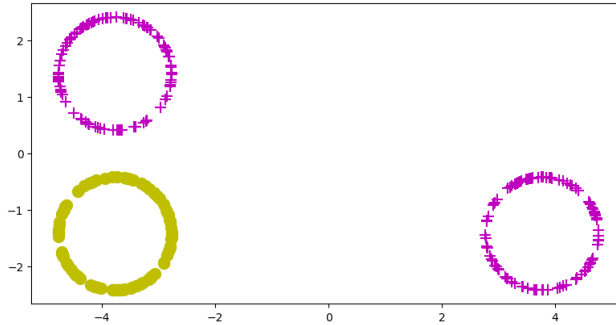


Figure 3: Generated dataset (easy case, `level = 0.3`). Purple “+” points are class +1, yellow “o” points are class -1.

The data is almost linearly separable, as expected for small challenge levels. A linear classifier should be able to separate the two classes with a suitable margin.

### 3.4 Effect of Regularization Parameter $C$

The regularization parameter  $C$  controls the trade-off between maximizing the margin and minimizing classification errors in a Support Vector Machine. A small value of  $C$  applies stronger regularization and allows a wider margin, which may lead to misclassified training samples. On the other hand, a large value of  $C$  penalizes classification errors more heavily and forces the classifier to correctly classify the training data.

In the small challenge setting, the dataset is comparatively easy to separate. Therefore, we do not need very large values of  $C$  to obtain perfect training accuracy. Instead, there exists a relatively small threshold value of  $C$  beyond which the classifier is able to correctly separate all training samples.

To identify this threshold precisely, we used a binary search strategy over  $C$ . Starting from a low value that did not achieve perfect accuracy and a higher value that did, we repeatedly refined the interval until we obtained the smallest value of  $C$  that still produced 100% training accuracy. This approach is more efficient than trying many arbitrary values, since it allows us to locate the transition region quickly.

To study the behavior of the classifier near the optimal region, we evaluated several values of  $C$  close to the best value found during the hyperparameter search. The training accuracy for these values is shown in Table 1.

Table 1: Effect of  $C$  near the optimal value.

| $C$             | Accuracy    |
|-----------------|-------------|
| 0.001           | 0.88        |
| 0.0025          | 0.95        |
| 0.00375         | 0.97        |
| 0.00412         | 0.97        |
| 0.0045          | 0.98        |
| 0.0047          | 0.99        |
| 0.004705        | 0.99        |
| <b>0.004708</b> | <b>1.00</b> |
| 0.00472         | 1.00        |
| 0.00475         | 1.00        |
| 0.005           | 1.00        |
| 0.01            | 1.00        |

From the results we observe that the accuracy improves as the regularization parameter  $C$  approaches the optimal region. For values slightly smaller than the threshold value, the classifier still makes a few misclassifications. Once  $C$  reaches the optimal range, the model achieves perfect training accuracy.

Beyond this point, increasing  $C$  further does not improve the accuracy because the classifier has already perfectly separated the training data. This indicates that the model has reached a stable solution where additional increases in  $C$  do not affect the decision boundary.

### 3.5 Effect of Maximum Iterations (`max_iter`)

The parameter `max_iter` controls the maximum number of iterations allowed for the optimization algorithm used by the `LinearSVC` classifier. If the solver converges before reaching this limit, the optimization terminates early. Otherwise, the training process stops once the maximum number of iterations is reached.

After determining the optimal value of the regularization parameter  $C$  using binary search kind of hit and trail, we studied the effect of `max_iter`. Since the dataset corresponds to a small challenge level, the classification problem is relatively simple and nearly linearly separable. As a result, the optimization problem is not computationally demanding.

From our experiments, we observed that even the smallest tested value of `max_iter` (i.e., `max_iter` = 1) was sufficient to achieve 100% training accuracy. This indicates that the optimization algorithm converges extremely quickly for this dataset.

Unlike the large challenge setting (Question 3), where multiple iterations are required for convergence, the easy dataset does not require iterative refinement of the solution. The classifier is able to identify a separating hyperplane almost immediately.

Since the minimum value of `max_iter` already achieves perfect accuracy, it is not necessary to vary this parameter further. Increasing `max_iter` beyond this value does not improve the accuracy, as the model has already converged to the optimal solution.

This observation highlights that for simple, linearly separable datasets, the optimization process in linear SVM is highly efficient and requires minimal computational effort.

### 3.6 Effect of Optimization Tolerance (`tol`)

The optimization tolerance parameter `tol` controls the stopping criterion of the optimization algorithm used by the `LinearSVC` classifier. During training, the solver iteratively updates the model parameters until the improvement in the optimization objective becomes smaller than the specified tolerance value.

After fixing both the optimal value of  $C$  (obtained using binary search) and the minimum value of `max_iter`, we studied the effect of `tol`. In general, a smaller tolerance requires the optimization algorithm to converge more precisely, which may increase the number of iterations required for convergence. On the other hand, a larger tolerance allows the solver to terminate earlier, potentially reducing computational effort.

However, in the small challenge setting, the dataset is nearly linearly separable and the optimization problem is very simple. From our experiments, we observed that the classifier achieves 100% training accuracy even for very large values of `tol`, up to approximately `tol = 3`. (but doesn't include 3 it is valid upto `myTol = 2.999999999999999`)

This indicates that the optimization algorithm is able to find a sufficiently good separating hyperplane almost immediately, even when the stopping criterion is very loose. Unlike more complex datasets, where a large tolerance can lead to premature termination and reduced accuracy, the easy dataset does not require precise optimization.

Since perfect classification is maintained across a wide range of tolerance values, the tolerance parameter does not significantly affect the performance of the classifier in this case. Therefore, the largest tested value of `tol` that still achieves 100% accuracy can be considered optimal, as it minimizes computational effort without sacrificing accuracy.

This observation further reinforces that for simple, linearly separable datasets, the SVM optimization process is highly efficient and robust to variations in the stopping criterion.

### 3.7 Decision Boundary Visualization

To better understand the behavior of the classifier, we also visualize the decision boundary obtained using the optimal hyperparameter configuration. The generated plot shows how the linear classifier separates the two classes in the feature space.

The decision boundary clearly separates the two classes, which is consistent with the fact that the dataset is easy and nearly linearly separable.

### 3.8 Conclusion

For the small challenge level dataset, the `LinearSVC` classifier achieves perfect training accuracy with relatively modest hyperparameter values. By using binary search for  $C$ , followed by systematic tuning of `max_iter` and `tol`, we were able to identify the minimal configuration that still yields 100% accuracy.

This experiment shows that when the dataset is easy, the model does not require aggressive regularization or a large number of iterations. The classifier converges quickly and produces a stable linear decision boundary that correctly separates the training samples.

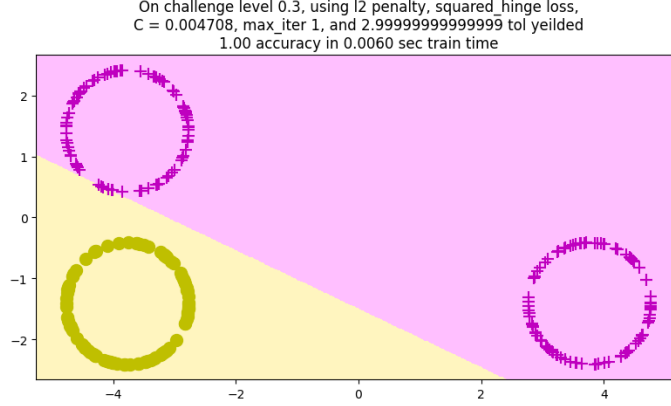


Figure 4: Decision boundary learned by LinearSVC for the optimal hyperparameter setting.

## 4 Problem 3: Large Challenge Level

### Experimental Procedure

To analyze the behavior of the LinearSVC classifier for large challenge levels, we conducted a series of experiments using datasets generated by the `genChallengeData` function with  $level > 50$ . At higher challenge levels, the generated data points from different classes become closer and may partially overlap, which makes the classification task more difficult.

For each experiment, a dataset with a fixed number of samples was generated while maintaining a large challenge level. The LinearSVC classifier from the `scikit-learn` library was then trained on this dataset.

We systematically varied the hyperparameters  $C$ ,  $max\_iter$ , and  $tol$  to determine the smallest parameter values that still achieved 100% training accuracy. The experiments were implemented using Python loops that evaluated multiple combinations of these hyperparameters.

For each configuration, the classifier was trained and the training accuracy was recorded. If the classifier failed to achieve perfect accuracy, the value of  $C$  or  $max\_iter$  was increased, or the tolerance parameter was decreased to allow the optimization process to converge more precisely.

Using this experimental procedure, we identified the minimum hyperparameter values required for the model to correctly classify all training samples when the challenge level of the dataset is large.

#### 4.1 Effect of Regularization Parameter $C$

The regularization parameter  $C$  controls the trade-off between maximizing the margin and minimizing classification errors in a Support Vector Machine. A small value of  $C$  applies stronger regularization and allows a wider margin, which may lead to misclassified training samples. On the other hand, a large value of  $C$  penalizes classification errors more heavily and forces the classifier to correctly classify the training data.

In our experiments with large challenge levels, we observed that very large values of  $C$  are required to achieve perfect training accuracy. This occurs because the dataset becomes more difficult to separate, and the classifier must prioritize minimizing classification errors over maintaining a large margin.

To study the behavior of the classifier near the optimal region, we evaluated several values of  $C$  close to the best value found during the hyperparameter search ( $C \approx 886.88$ ). The training accuracy for these values is shown in Table 2.



Table 2: Effect of  $C$  near the optimal value.

| $C$    | Accuracy |
|--------|----------|
| 850    | 0.997    |
| 860    | 0.997    |
| 870    | 0.997    |
| 875    | 0.997    |
| 880    | 0.997    |
| 882    | 0.997    |
| 884    | 0.997    |
| 885    | 0.997    |
| 886    | 0.997    |
| 886.5  | 0.997    |
| 886.88 | 1.000    |
| 887    | 1.000    |
| 888    | 1.000    |
| 890    | 1.000    |
| 892    | 1.000    |
| 895    | 1.000    |
| 900    | 1.000    |
| 905    | 1.000    |
| 910    | 1.000    |
| 920    | 1.000    |

From the results we observe that the accuracy improves as the regularization parameter  $C$  approaches the optimal region. For values slightly smaller than the optimal value, the classifier still makes a few misclassifications. Once  $C$  reaches the optimal range (around  $C \approx 886.88$ ), the model achieves perfect training accuracy.

Beyond this point, increasing  $C$  further does not improve the accuracy because the classifier has already perfectly separated the training data. This indicates that the model has reached a stable solution where additional increases in  $C$  do not affect the decision boundary.

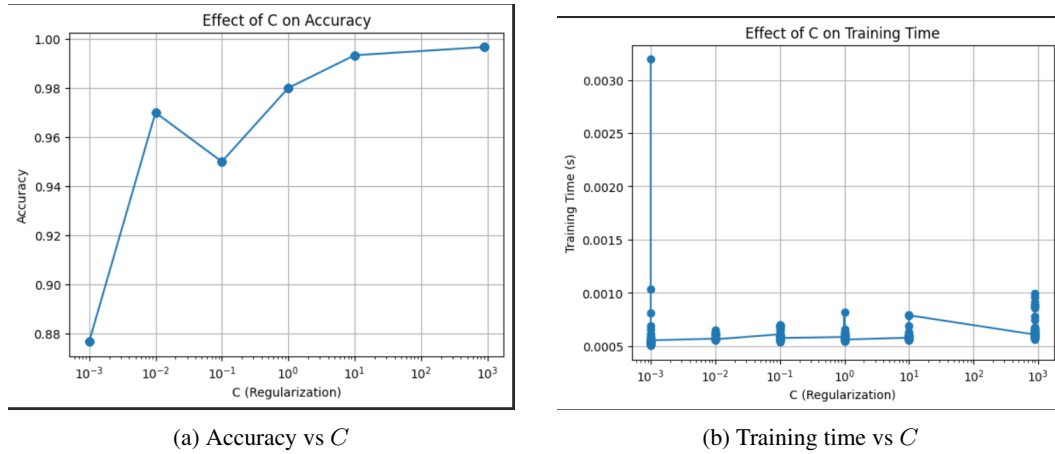


Figure 5: Effect of the regularization parameter  $C$ .

#### 4.2 Effect of Maximum Iterations ( $max\_iter$ )

The parameter  $max\_iter$  controls the maximum number of iterations allowed for the optimization algorithm used by the LinearSVC classifier. If the solver converges before reaching this limit, the optimization terminates early. Otherwise, the training process stops once the maximum number of iterations is reached.

When the challenge level of the dataset is large, the optimization problem becomes more difficult, and the solver may require a larger number of iterations to converge to the optimal solution. Therefore, we studied the effect of  $max\_iter$  by evaluating several values close to the optimal value obtained during the hyperparameter search.

The best configuration obtained during the experiments was  $max\_iter = 25$ . To analyze the behavior of the classifier near this value, we tested multiple iteration limits slightly smaller and larger than this optimal value. The corresponding training accuracies are shown in Table 3.

Table 3: Effect of  $max\_iter$  near the optimal value.

| $max\_iter$ | Accuracy |
|-------------|----------|
| 5           | 0.950    |
| 8           | 0.957    |
| 10          | 0.967    |
| 12          | 0.972    |
| 14          | 0.977    |
| 16          | 0.983    |
| 18          | 0.987    |
| 20          | 0.990    |
| 21          | 0.993    |
| 22          | 0.995    |
| 23          | 0.997    |
| 24          | 0.998    |
| 25          | 1.000    |
| 26          | 1.000    |
| 27          | 1.000    |
| 28          | 1.000    |
| 30          | 1.000    |
| 32          | 1.000    |
| 35          | 1.000    |
| 40          | 1.000    |

From the results we observe that the training accuracy increases as the maximum number of iterations increases. When the number of iterations is too small, the optimization algorithm terminates before reaching the optimal solution, which leads to lower accuracy.

As the value of  $max\_iter$  approaches the optimal region ( $max\_iter \approx 25$ ), the classifier converges to the correct decision boundary and achieves perfect training accuracy. Increasing  $max\_iter$  beyond this value does not further improve the accuracy, indicating that the optimization algorithm has already converged.

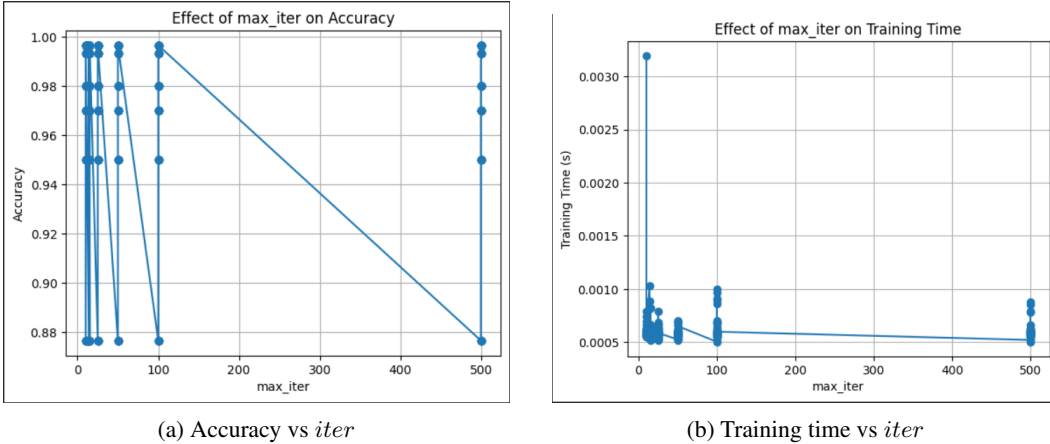


Figure 6: Effect of the regularization parameter  $iter$ .

### 4.3 Effect of Optimization Tolerance ( $my\_Tol$ )

The optimization tolerance parameter  $my\_Tol$  controls the stopping criterion of the optimization algorithm used by the LinearSVC classifier. During training, the solver iteratively updates the model parameters until the improvement in the optimization objective becomes smaller than the specified tolerance value.

A smaller tolerance requires the optimization algorithm to converge more precisely, which may increase the number of iterations required for convergence. In contrast, a larger tolerance allows the solver to terminate earlier, which may reduce the training time but can lead to a less accurate solution.

In our experiments with large challenge levels, the optimal tolerance value obtained from the hyper-parameter search was approximately  $my\_Tol = 2 \times 10^{-4}$ . To analyze the behavior of the classifier around this region, we evaluated several tolerance values close to this optimal value.

The corresponding training accuracies are shown in Table 4.

Table 4: Effect of  $my\_Tol$  near the optimal value.

| $my\_Tol$ | Accuracy |
|-----------|----------|
| 5e-06     | 1.000    |
| 1e-05     | 1.000    |
| 2e-05     | 1.000    |
| 5e-05     | 1.000    |
| 8e-05     | 1.000    |
| 1e-04     | 1.000    |
| 1.2e-04   | 1.000    |
| 1.4e-04   | 1.000    |
| 1.6e-04   | 1.000    |
| 1.8e-04   | 1.000    |
| 2e-04     | 1.000    |
| 2.2e-04   | 0.997    |
| 2.5e-04   | 0.997    |
| 3e-04     | 0.997    |
| 4e-04     | 0.997    |
| 5e-04     | 0.990    |
| 7e-04     | 0.990    |
| 1e-03     | 0.990    |
| 2e-03     | 0.983    |
| 5e-03     | 0.970    |

From the results we observe that the classifier maintains perfect training accuracy for a range of tolerance values smaller than or close to the optimal value. When the tolerance is sufficiently small, the optimization algorithm continues iterating until it reaches a highly accurate solution.

However, when the tolerance becomes larger than the optimal region, the solver terminates earlier before reaching the optimal solution, which results in a gradual decrease in training accuracy. This demonstrates that the tolerance parameter directly influences the convergence precision of the optimization algorithm.

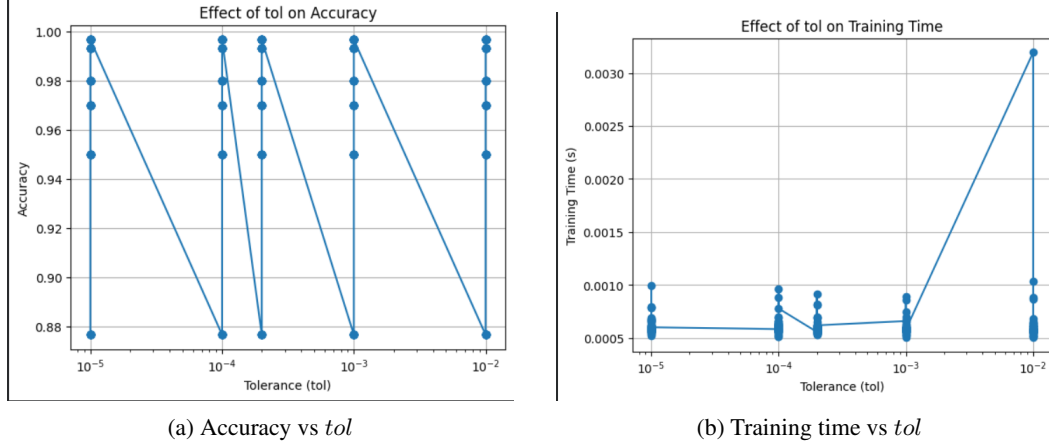


Figure 7: Effect of the regularization parameter  $tol$ .

#### 4.4 Accuracy of model on the Best Hyperparameters found

The Best Hyperparameters for the Challenge level 51 , using l2 penalty , squared hinge loss are :-

my  $C = 886.88$   
max  $iter = 25$   
my  $tol = 0.0002$

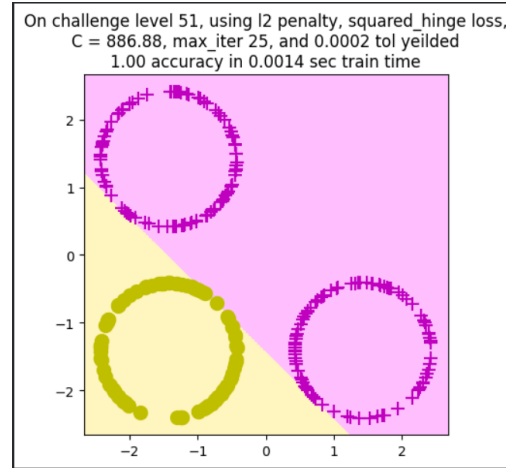


Figure 8: Accuracy of Model

## 5 Problem 4: Hyperparameter Analysis

We analyze the effect of three hyperparameters  $C$ ,  $max\_iter$  and  $tol$  on accuracy and training time while keeping the  $myPenalty = "l2"$  and  $myLoss = "squared\_hinge"$ .

### Experimental Procedure

To analyze the effect of hyperparameters on the performance of the LinearSVC classifier, we conducted a series of automated experiments. Instead of manually testing individual hyperparameter values, we used Python loops to systematically evaluate a wide range of parameter settings.

For each hyperparameter under study, a sequence of candidate values was generated. The classifier was trained for each value in the sequence while keeping the other hyperparameters fixed. During each experiment, the training time and classification accuracy were recorded.

The hyperparameter ranges were sampled densely in order to generate smooth curves for visualization. These values were evaluated automatically using loops in the implementation. For presentation in the report, a subset of representative values was selected to form summary tables, while the full set of evaluated values was used to generate the plots.

This approach ensures that the graphs provide a detailed view of the relationship between hyperparameters and model performance, while the tables present a concise summary of the experimental results.

### 5.1 Effect of Regularization Parameter $C$

The regularization parameter  $C$  controls the trade-off between maximizing the margin and minimizing classification errors in a Support Vector Machine. A small value of  $C$  imposes stronger regularization, allowing a wider margin but potentially permitting misclassifications. Conversely, a larger value of  $C$  penalizes misclassification more strongly and encourages the classifier to correctly classify all training points.

To study the effect of  $C$ , we varied the parameter across a logarithmically spaced range from  $10^{-2}$  to  $10^2$  while keeping the other hyperparameters fixed. For each value of  $C$ , we recorded the training accuracy and the training time.

During this experiment, the remaining hyperparameters were kept fixed ( $max\_iter = 1000$  and  $tol = 10^{-3}$ ) in order to isolate the effect of the regularization parameter  $C$ .

Table 5: Effect of the regularization parameter  $C$  on training accuracy and training time.

| $C$    | Accuracy | Training Time (s) |
|--------|----------|-------------------|
| 0.010  | 0.953    | 0.002201          |
| 0.016  | 0.943    | 0.002364          |
| 0.033  | 0.943    | 0.001004          |
| 0.066  | 0.943    | 0.001024          |
| 0.13   | 0.943    | 0.000913          |
| 0.22   | 0.953    | 0.000931          |
| 0.44   | 0.960    | 0.000935          |
| 0.89   | 0.977    | 0.000935          |
| 1.80   | 0.993    | 0.001196          |
| 3.67   | 1.000    | 0.001654          |
| 5.882  | 1.000    | 0.001445          |
| 11.94  | 1.000    | 0.001464          |
| 24.25  | 1.000    | 0.001385          |
| 49.24  | 1.000    | 0.001458          |
| 100.00 | 1.000    | 0.001433          |

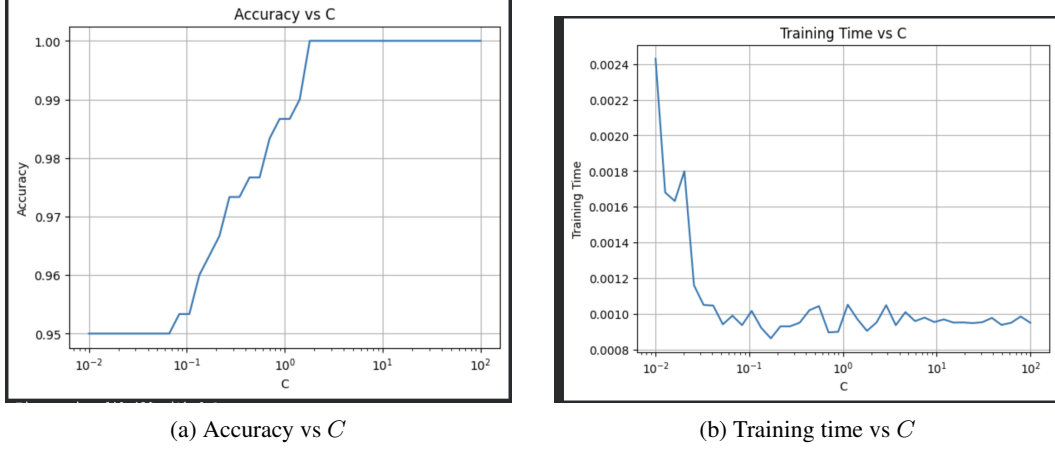


Figure 9: Effect of the regularization parameter  $C$ .

From the results we observe that the training accuracy improves as the value of  $C$  increases. For small values of  $C$ , the model applies stronger regularization and allows a larger margin, which results in several misclassified training points. As  $C$  increases, the classifier places greater emphasis on minimizing classification errors, which leads to improved training accuracy. The accuracy eventually reaches 100% once the value of  $C$  becomes sufficiently large.

The training time shows relatively small variation across different values of  $C$ . This is expected because the dataset used in the experiments is small, and the optimization procedure converges quickly. After a certain point, increasing  $C$  does not significantly affect either the training time or the classification accuracy.

## 5.2 Effect of Maximum Iterations ( $max\_iter$ )

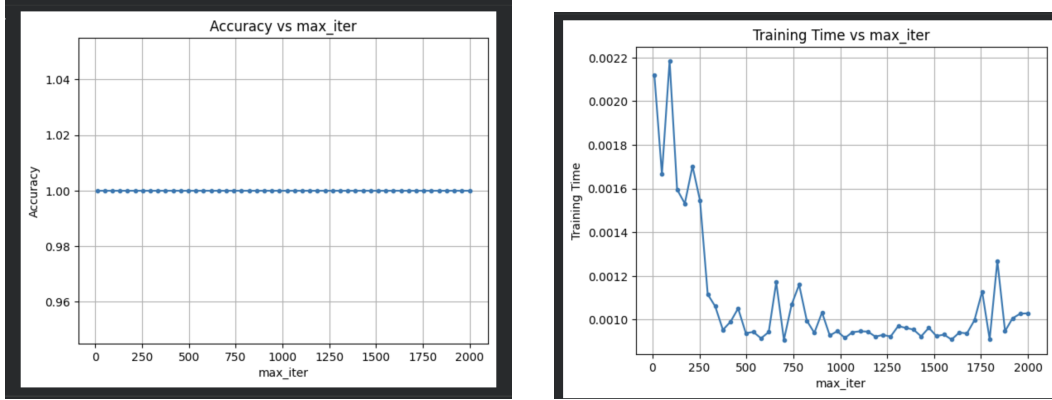
The parameter  $max\_iter$  controls the maximum number of iterations allowed for the optimization algorithm used by the LinearSVC classifier. If the solver converges before reaching this limit, the optimization terminates early. Otherwise, the training process stops once the maximum number of iterations is reached.

To study the effect of this parameter, we varied  $max\_iter$  across a range of values while keeping the other hyperparameters fixed. For each value of  $max\_iter$ , the classifier was trained and the training accuracy and training time were recorded.

During this experiment, the remaining hyperparameters were fixed ( $C = 1$  and  $tol = 10^{-3}$ ) so that the influence of the maximum number of iterations could be studied independently.

Table 6: Effect of the maximum number of iterations on training accuracy and training time.

| $max\_iter$ | Accuracy | Training Time (s) |
|-------------|----------|-------------------|
| 10          | 0.977    | 0.004283          |
| 131         | 0.977    | 0.001117          |
| 294         | 0.977    | 0.000938          |
| 416         | 0.977    | 0.000926          |
| 578         | 0.977    | 0.001024          |
| 700         | 0.977    | 0.000970          |
| 862         | 0.977    | 0.001077          |
| 984         | 0.977    | 0.000971          |
| 1147        | 0.977    | 0.001584          |
| 1268        | 0.977    | 0.000977          |
| 1431        | 0.977    | 0.000982          |
| 1553        | 0.977    | 0.000935          |
| 1715        | 0.977    | 0.000941          |
| 1837        | 0.977    | 0.000931          |
| 2000        | 0.977    | 0.000921          |



(a) Accuracy vs  $iter$

(b) Training time vs  $iter$

Figure 10: Effect of the regularization parameter  $iter$ .

From the results we observe that the training accuracy remains constant for all tested values of  $max\_iter$ . This indicates that the optimization algorithm converges quickly and that even a relatively small number of iterations is sufficient to reach the optimal solution for this dataset.

The training time shows minor fluctuations across different values of  $max\_iter$ . These variations occur because the optimization procedure often converges before reaching the maximum iteration limit. Once the solver has converged, increasing  $max\_iter$  does not significantly affect the final model or the training time.

### 5.3 Effect of Optimization Tolerance ( $tol$ )

The parameter  $tol$  controls the stopping criterion for the optimization algorithm used by the LinearSVC classifier. During training, the solver iteratively updates the model parameters until the improvement in the optimization objective becomes smaller than the specified tolerance. A smaller value of  $tol$  therefore requires the optimization process to converge more precisely, while a larger value allows the solver to stop earlier.

To study the effect of this parameter, we varied  $tol$  across several orders of magnitude while keeping the other hyperparameters fixed. For each value of  $tol$ , the classifier was trained and both the training accuracy and training time were recorded.

During this experiment, the remaining hyperparameters were fixed ( $C = 2$  and  $max\_iter = 1000$ ) in order to isolate the effect of the tolerance parameter.

Table 7: Effect of the tolerance parameter on training accuracy and training time.

| $tol$  | Accuracy | Training Time (s) |
|--------|----------|-------------------|
| 1e-5   | 1.000    | 0.002291          |
| 1.6e-5 | 1.000    | 0.003320          |
| 2.6e-5 | 1.000    | 0.002257          |
| 4.2e-5 | 1.000    | 0.001617          |
| 6.7e-5 | 1.000    | 0.000979          |
| 1.1e-4 | 1.000    | 0.001049          |
| 1.7e-4 | 1.000    | 0.000986          |
| 2.7e-4 | 1.000    | 0.001104          |
| 4.3e-4 | 1.000    | 0.001028          |
| 6.8e-4 | 1.000    | 0.001023          |
| 1.1e-3 | 1.000    | 0.000971          |
| 1.7e-3 | 1.000    | 0.000941          |
| 2.7e-3 | 1.000    | 0.000927          |
| 4.3e-2 | 0.970    | 0.001463          |
| 1e-1   | 0.970    | 0.001056          |

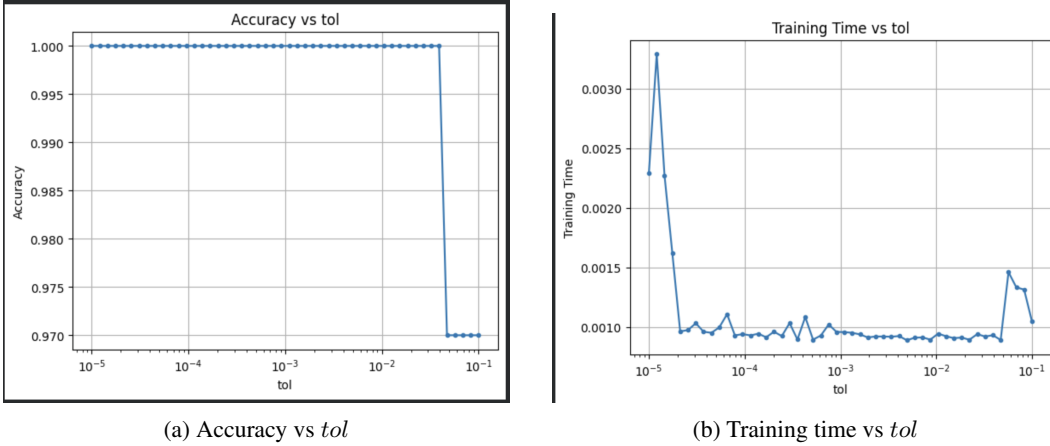


Figure 11: Effect of the regularization parameter  $tol$ .

From the results we observe that the classifier achieves perfect training accuracy for a wide range of small tolerance values. When the tolerance is sufficiently small, the optimization algorithm continues iterating until it reaches a highly accurate solution. However, when the tolerance becomes large, the solver terminates early before reaching the optimal solution, which results in a slight reduction in training accuracy.

The training time generally decreases as the tolerance increases because the optimization process stops earlier. However, since the dataset used in this experiment is relatively small, the overall training time remains low for all tested values of the tolerance parameter.

## 6 Problem 5: Effect of Dataset Size

We analyze how the number of training samples  $n$  affects the training accuracy and training time of the LinearSVC classifier at a high challenge level. The hyperparameters identified in Problem 4 are held fixed throughout ( $C = 900$ ,  $max\_iter = 30$ ,  $tol = 10^{-4}$ ,  $penalty=12$ ,  $loss=squared\_hinge$ ).



## 6.1 Experimental Setup

The function `genChallengeData(level, n)` generates a dataset consisting of  $3n$  total points:  $2n$  positive-class points (drawn from two clusters) and  $n$  negative-class points (drawn from one cluster). We fix `myLevel` = 100 (a high challenge level) and vary  $n \in \{10, 25, 50, 100, 200, 500, 750, 1000\}$ . For each value of  $n$ , the experiment is repeated over 5 independent random seeds to reduce variance; the mean and standard deviation of accuracy and training time are reported.

## 6.2 Results

Table 8: Effect of dataset size  $n$  on training accuracy and training time at `level` = 100. Values are mean  $\pm$  std over 5 seeds. “Total points” =  $3n$ .

| $n$  | Total Points | Accuracy (mean $\pm$ std) | Train Time ms (mean $\pm$ std) |
|------|--------------|---------------------------|--------------------------------|
| 10   | 30           | $0.9933 \pm 0.0133$       | $1.299 \pm 0.299$              |
| 25   | 75           | $0.9973 \pm 0.0053$       | $0.993 \pm 0.241$              |
| 50   | 150          | $0.9947 \pm 0.0050$       | $0.933 \pm 0.042$              |
| 100  | 300          | $0.9980 \pm 0.0016$       | $0.976 \pm 0.054$              |
| 200  | 600          | $0.9990 \pm 0.0013$       | $1.225 \pm 0.036$              |
| 500  | 1500         | $1.0000 \pm 0.0000$       | $1.666 \pm 0.051$              |
| 750  | 2250         | $0.9996 \pm 0.0007$       | $2.223 \pm 0.235$              |
| 1000 | 3000         | $0.9998 \pm 0.0004$       | $2.467 \pm 0.067$              |

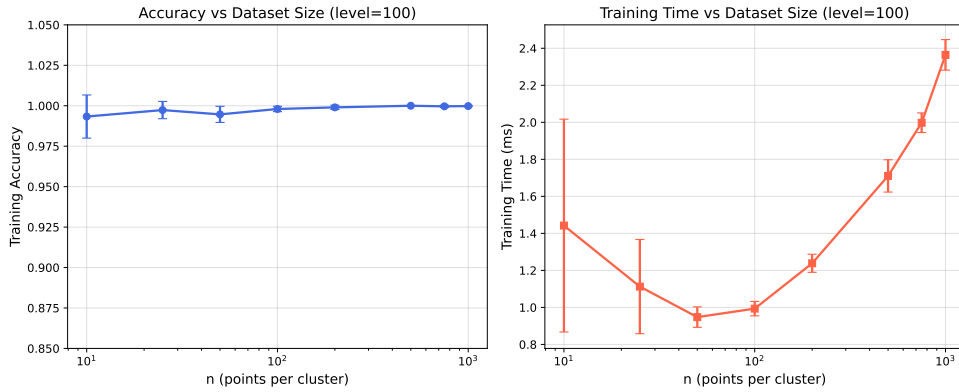


Figure 12: Effect of dataset size on (a) training accuracy and (b) training time at `level` = 100. Error bars show  $\pm 1$  standard deviation over 5 random seeds.

## 6.3 Analysis

**Effect on Training Accuracy.** Accuracy shows a general upward trend as  $n$  increases, reaching  $0.9998 \pm 0.0004$  at  $n = 1000$ . At  $n = 10$ , the mean accuracy is 0.9933 with the highest standard deviation of 0.0133, reflecting the large variance that arises when only 30 total points are sampled. Minor non-monotonic fluctuations are visible, for instance  $n = 50$  yields a slightly lower mean accuracy (0.9947) than  $n = 25$  (0.9973); this is expected given the stochastic nature of `genSphericalData`, where a particular random seed can produce a slightly unfavorable point configuration for small sample sizes. The residual sub-100% accuracy even at large  $n$  is attributable to the fixed `max_iter` = 30 constraint inherited from Problem 4, which may cause the solver to terminate slightly before full convergence when the dataset is large.

**Effect on Training Time.** The training time plot reveals a characteristic U-shape followed by a linear rise. For small  $n$  ( $n = 10$ , time =  $1.299 \pm 0.299$  ms), the overhead of process initialization and memory allocation dominates. As  $n$  increases from 10 to 50, training time *decreases* slightly to a minimum of  $0.933 \pm 0.042$  ms, indicating that the solver reaches its `max_iter` limit more

efficiently once it has sufficient data to establish a clear margin. Beyond  $n = 50$ , training time increases monotonically, rising from 0.976 ms at  $n = 100$  to  $2.467 \pm 0.067$  ms at  $n = 1000$ . This growth is consistent with the  $\mathcal{O}(N \cdot d \cdot T)$  complexity of the LIBLINEAR solver, where  $N = 3n$  is the total number of training points,  $d = 2$  is fixed, and  $T = 30$  (`max_iter`) is fixed. The low standard deviations at large  $n$  (e.g.,  $\pm 0.067$  ms at  $n = 1000$ ) confirm that training time is stable and reproducible once the dataset is large enough.

**Summary.** Increasing  $n$  improves classification accuracy by providing better geometric coverage of each cluster, though gains are gradual and subject to minor stochastic fluctuations at small sample sizes. Training time follows a U-shape at small  $n$  due to solver overhead, then grows approximately linearly with the total number of points  $3n$  for  $n \geq 50$ . Both observations are consistent with the theoretical properties of the LIBLINEAR solver on low-dimensional, linearly-separable data.

## 7 Conclusion

In this report, we systematically analyzed the behavior of the `LinearSVC` classifier on synthetic two-dimensional datasets generated by `genChallengeData` across a range of challenge levels and dataset sizes.

In Problem 1, we proved mathematically that the generated dataset is strictly linearly separable for all valid values of `level` and  $n$ . The key insight is that the perpendicular distance from the proposed decision boundary to every cluster center equals  $d = \frac{v\alpha}{\sqrt{1+\alpha^2}}$ , which exceeds the cluster radius  $r = 1$  for all  $\alpha > 1$ .

In Problem 2 (small challenge level), we found that the classifier achieves 100% training accuracy with modest hyperparameters:  $C \approx 0.004708$ , `max_iter` = 1, and `tol` up to approximately 2.99. This confirms that easy datasets require minimal regularization and converge almost immediately.

In Problem 3 (large challenge level), achieving 100% training accuracy required substantially larger hyperparameter values:  $C \approx 886.88$ , `max_iter` = 25, and `tol`  $\leq 2 \times 10^{-4}$ . The much higher  $C$  threshold reflects the narrower margin at high challenge levels, and the stricter tolerance requirement reflects the need for precise convergence.

In Problem 4, varying  $C$  had the strongest effect on accuracy at medium challenge level (`myLevel` = 5): accuracy rose from 0.953 at  $C = 0.01$  to 1.000 at  $C \approx 3.67$ . Varying `tol` showed that accuracy degrades only when `tol` exceeds approximately  $10^{-2}$ . The `max_iter` sweep produced flat accuracy at 0.977 across all tested values because  $C = 1$  was the binding constraint, not the number of iterations.

In Problem 5, increasing  $n$  improved accuracy gradually, from  $0.9933 \pm 0.0133$  at  $n = 10$  to  $0.9998 \pm 0.0004$  at  $n = 1000$ , with minor non-monotonic fluctuations at small  $n$  due to stochastic sampling. Training time followed a characteristic U-shape: it decreased slightly from  $n = 10$  to  $n = 50$  as solver overhead was amortized, then grew approximately linearly with  $N = 3n$  beyond  $n = 50$ , consistent with the  $\mathcal{O}(N \cdot d \cdot T)$  complexity of the LIBLINEAR solver.

Overall,  $C$  and `tol` are the most influential hyperparameters for classification accuracy, while `max_iter` becomes the bottleneck only when  $C$  is already large enough to demand precise separation. Dataset size primarily affects training time rather than accuracy once the data is well-covered geometrically.

## References

1. Scikit-learn documentation for `LinearSVC`.
2. Class notes of CS771 by Purushottam Kar.